

Singleton Pattern in JavaScript

What is the Singleton Pattern?

The **Singleton Pattern** is a creational design pattern that ensures a class has **only one instance** throughout the lifetime of an application and provides a **global access point** to that instance.

In simple terms:

"Create an object once, reuse the same object everywhere."

Instead of allowing code to create multiple copies of the same object:

```
const database1 = new Database();
const database2 = new Database();

console.log(database1 === database2); // false
```

The Singleton Pattern ensures:

```
const database1 = Database.getInstance();
const database2 = Database.getInstance();

console.log(database1 === database2); // true
```

Both variables point to the **same object in memory**.

Basic Singleton Example in JavaScript

Database Connection Example

A common use case is managing a database connection.

You usually only want **one database connection manager** because:

- Creating multiple connections wastes resources.
- It can exceed database connection limits.
- Different parts of the application should share the same connection.

```
class Database {
  constructor() {
```

```

    // Check if an instance already exists.
    // If it does, return that existing instance instead
    // of creating a new object.
    if (Database.instance) {
        return Database.instance;
    }

    // Store database configuration.
    this.connection = "Connected to database";

    // Save this object as the single instance.
    // Future calls will return this same object.
    Database.instance = this;
}

// Example method that uses the database connection
query(sql) {
    console.log(`Running query: ${sql}`);
    console.log(this.connection);
}
}

// Creating objects
const database1 = new Database();
const database2 = new Database();

console.log(database1 === database2);
// true

database1.query("SELECT * FROM users");

```

How This Works Step-by-Step

1. First Object Creation

```
const database1 = new Database();
```

JavaScript runs the constructor:

```
constructor() {  
  
}
```

At this point:

```
Database.instance
```

does not exist.

So the constructor continues:

```
Database.instance = this;
```

The first object is stored.

Memory:

```
Database.instance  
  |  
  v  
+-----+  
| Database Object |  
| connection      |  
+-----+  
  
database1  
  |  
  v  
+-----+  
| Database Object |  
+-----+
```

2. Second Object Creation

```
const database2 = new Database();
```

The constructor runs again.

This time:

```
if (Database.instance)
```

is true.

The constructor immediately returns:

```
return Database.instance;
```

No new object is created.

Memory:

```
Database.instance
  |
  +-----+
  |
database1 -----+
  |
database2 -----+
  |
                Same Object
```

Therefore:

```
database1 === database2
```

returns:

```
true
```

A More Realistic JavaScript Singleton Example

Application Configuration Manager

Large applications often need a single place to store configuration:

- API URLs
- Environment variables
- Application settings
- Feature flags

```

class Config {

  constructor() {

    // Prevent multiple Config objects
    if (Config.instance) {
      return Config.instance;
    }

    // Application settings
    this.settings = {
      apiUrl: "https://api.example.com",
      theme: "dark",
      language: "en"
    };

    // Save singleton instance
    Config.instance = this;
  }

  get(key) {
    return this.settings[key];
  }

  set(key, value) {
    this.settings[key] = value;
  }
}

// First part of application
const config1 = new Config();

console.log(config1.get("theme"));
// dark

// Another part of application
const config2 = new Config();

config2.set("theme", "light");

// Because both variables reference

```

```
// the same object, config1 changes too.  
  
console.log(config1.get("theme"));  
// light  
  
console.log(config1 === config2);  
// true
```

Singleton Using a Module (Common Modern JavaScript Approach)

In JavaScript modules, the module system itself provides singleton behaviour.

Example:

logger.js

```
class Logger {  
  
  constructor() {  
    this.logs = [];  
  }  
  
  add(message) {  
    this.logs.push(message);  
    console.log(message);  
  }  
  
  count() {  
    return this.logs.length;  
  }  
}  
  
// Create one instance  
const logger = new Logger();  
  
// Export the same instance  
export default logger;
```

app.js

```
import logger from "../logger.js";

logger.add("User logged in");
```

dashboard.js

```
import logger from "../logger.js";

console.log(logger.count());
```

Both files receive the **same logger object**.

The module system caches imported modules:

```
logger.js
  |
  v
+-----+
| Logger Object |
| logs[]       |
+-----+
  |
  |
+-----+-----+
|                   |
|                   |
+-----+-----+
app.js    dashboard.js
```

Common Uses of Singleton Pattern in JavaScript

1. Database Connections

Example:

```
Database.getInstance()
```

Used in:

- Node.js applications
- MongoDB connections
- PostgreSQL connection pools

Problem solved:

Prevent creating multiple expensive database connections.

2. Application Configuration

Example:

```
Config.get("API_URL")
```

Used for:

- Environment variables
- Feature flags
- Application settings

Problem solved:

Maintain one consistent source of configuration.

3. Logging Systems

Example:

```
Logger.log("Error occurred");
```

Used for:

- Error tracking
- Audit logs
- Debugging

Problem solved:

Prevent multiple loggers writing inconsistent information.

4. State Management

Examples:

- Redux store
- Application state managers

Problem solved:

Keep one central state object shared across components.

5. Caching

Example:

```
Cache.get("users");
```

Problem solved:

Ensure all parts of an application use the same cached data.

Problems Singleton Solves

Problem 1: Duplicate Objects

Without Singleton:

```
const settings1 = new Settings();  
const settings2 = new Settings();
```

You now have two separate objects.

Changes to one do not affect the other.

Singleton:

```
const settings1 = Settings.getInstance();  
const settings2 = Settings.getInstance();
```

One shared object.

Problem 2: Resource Waste

Some objects are expensive:

- Database connections
- API clients
- File handlers
- WebSocket connections

Creating many copies wastes memory and resources.

Problem 3: Synchronisation Problems

Without Singleton:

```
Component A
  |
  v
Settings Object #1

Component B
  |
  v
Settings Object #2
```

They can contain different data.

Singleton:

```
Component A
  |
  v
Shared Object
  ^
  |
Component B
```

Everyone sees the same state.

Disadvantages of Singleton Pattern

Although useful, Singleton is sometimes criticised.

1. Creates Global State

Example:

```
Logger.logs
```

Any part of the application can modify it.

This can make debugging harder.

2. Makes Testing More Difficult

Example:

```
Database.instance
```

A test might accidentally reuse old data from another test.

3. Can Hide Dependencies

Example:

```
function saveUser(user){  
    Database.query(user);  
}
```

The function secretly depends on Database.

A cleaner approach might be dependency injection:

```
function saveUser(user, database){  
}
```

Singleton vs Normal Class

Normal Class	Singleton
Multiple objects allowed	Only one object exists
Independent state	Shared state
Used for general objects	Used for shared resources
Easier testing	Can complicate testing

When Should You Use Singleton?

Use Singleton when:

- ✓ You genuinely need exactly one shared instance
- ✓ Creating multiple instances is expensive
- ✓ The object represents a global resource

Examples:

- Database connection manager
- Logger
- Configuration manager
- Cache manager

Avoid Singleton when:

- ✗ You only want convenient global access
- ✗ The object contains lots of mutable state
- ✗ Dependency injection would be cleaner

Key Takeaway

The Singleton Pattern ensures:

```
One Class
  |
  v
One Instance
  |
  v
Shared Everywhere
```

In JavaScript, it is often implemented using:

1. A private/static stored instance inside a class.
2. ES modules exporting a single object instance.

It is mainly used to manage **shared resources and application-wide state** while preventing unnecessary duplicate objects.